Assignment 2

```
Name : Ravish Ranjan
Roll no. : 45
Course : MCA
Subject : Information Security
```

1. Write a small C program with an intentionally unsafe function (e.g., using gets() or strcpy() into a fixed-size stack buffer, with no bounds checking).

```c
#include <string.h>
#include <stdio.h>

void testing_function(const char* input) {
    char buffer[24];
    strcpy(buffer,input);

    printf("Buffered content : %s\n",buffer);
}

int main(int argc,char *args[]) {
    if (argc != 2) {
        printf("Wrong format\n\tUsage : %s <input>\n",args[0]);
    }

    testing_function(args[1]);

    return 0;
}
```

2. Compile it WITHOUT protection and run it with a normal input, then a deliberately oversized input to trigger a crash.

compiling program

```
ravish@ravishPC:../sem3/is ⎇ (main)> gcc -g -O0 -fno-stack-protector -no-
pie ./prog/a2.c -o ./prog/a2.out
```

checking output

```
ravish@ravishPC:../sem3/is ⎇ (main)> ls -l ./prog
total 24
-rw-rw-r-- 1 ravish ravish   353 Aug 18 12:18 a2.c
```

```
-rwxrwxr-x 1 ravish ravish 17480 Aug 18 12:24 a2.out
ravish@ravishPC:../sem3/is ⎇ (main)>
```

running non-overflow code

```
ravish@ravishPC:../sem3/is ⎇ (main)> ./prog/a2.out "Testing"
Buffered content : Testing
ravish@ravishPC:../sem3/is ⎇ (main)>
```

running overflow code

```
ravish@ravishPC:../sem3/is ⎇ (main)> ./prog/a2.out "Testing with
overflowing buffer with input"
Buffered content : Testing with overflowing buffer with input
Segmentation fault (core dumped)
ravish@ravishPC:../sem3/is ⎇ (main)>
```

3. Use gdb to inspect the crash: identify the overwritten return address / corrupted stack frame, and capture the register/backtrace state at the point of failure.

opening gdb

```
ravish@ravishPC:../sem3/is ⎇ (main)> gdb ./prog/a2.out
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later
<http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./prog/a2.out...
```

running overflow input

```
(gdb) run "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
Starting program: /home/ravish/work/sem3/is/prog/a2.out
```

```
"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Buffered content : AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

Program received signal SIGBUS, Bus error.
main (
    argc=<error reading variable: Cannot access memory at address
0x4141414141413d>,
    args=<error reading variable: Cannot access memory at address
0x41414141414131>) at ./prog/a2.c:19
19      }
```

checking backtrace

```
(gdb) bt
#0  main (
    argc=<error reading variable: Cannot access memory at address
0x4141414141413d>,
    args=<error reading variable: Cannot access memory at address
0x41414141414131>) at ./prog/a2.c:19
Backtrace stopped: Cannot access memory at address 0x41414141414149
```

checking debug frames

```
(gdb) info frame
Stack level 0, frame at 0x41414141414151:
 rip = 0x4011e6 in main (./prog/a2.c:19); saved rip = <not saved>
 Outermost frame: Cannot access memory at address 0x41414141414149
 source language c.
 Arglist at 0x41414141414141, args:
    argc=<error reading variable: Cannot access memory at address
0x4141414141413d>,
    args=<error reading variable: Cannot access memory at address
0x41414141414131>
 Locals at 0x41414141414141, Previous frame's sp is 0x41414141414151
Cannot access memory at address 0x41414141414141
```

checking memory space

```
(gdb) x/40gx $rsp
0x7fffffffd5b0: 0x00007fffffffd6e8 0x00000002ffffd6e8
0x7fffffffd5c0: 0x00007fffffffd660 0x00007ffff7c2a1ca
0x7fffffffd5d0: 0x00007fffffffd610 0x00007fffffffd6e8
0x7fffffffd5e0: 0x0000000200400040 0x0000000000401197
0x7fffffffd5f0: 0x00007fffffffd6e8 0xfda26ad23e359136
0x7fffffffd600: 0x0000000000000002 0x0000000000000000
```

```
0x7fffffffd610: 0x0000000000403e00 0x00007ffff7ffd000
0x7fffffffd620: 0xfda26ad239559136 0xfda27aa8d1f79136
0x7fffffffd630: 0x00007fff00000000 0x0000000000000000
0x7fffffffd640: 0x0000000000000000 0x0000000000000002
0x7fffffffd650: 0x00007fffffffd6e0 0xc985f0f1be396600
0x7fffffffd660: 0x00007fffffffd6c0 0x00007ffff7c2a28b
0x7fffffffd670: 0x00007fffffffd700 0x0000000000403e00
0x7fffffffd680: 0x00007fffffffd700 0x0000000000401197
0x7fffffffd690: 0x0000000000000000 0x0000000000000000
0x7fffffffd6a0: 0x0000000000401070 0x00007fffffffd6e0
0x7fffffffd6b0: 0x0000000000000000 0x0000000000000000
0x7fffffffd6c0: 0x0000000000000000 0x0000000000401095
0x7fffffffd6d0: 0x00007fffffffd6d8 0x0000000000000038
0x7fffffffd6e0: 0x0000000000000002 0x00007fffffffdb16
```

checking regsiter values (the hexa decimal format of "AAAAAAAA....." can be found in rbp register "0x4141414141414141")

```
(gdb) info registers rip rbp rsp
rip            0x401100            0x401100 <register_tm_clones+32>
rbp            0x4141414141414141  0x4141414141414141
rsp            0x7fffffffd5b0      0x7fffffffd5b0
```

4. Recompile the SAME program WITH protections enabled. Repeat the oversized input and show that the program now terminates safely (e.g., “stack smashing detected”) instead of executing arbitrary control flow.

compiling safe program

```
ravish@ravishPC:../sem3/is ⎇ (main)> gcc -g -O0 -fstack-protector-strong -
no-pie ./prog/a2.c -o ./prog/a2_pro.out
```

checking output

```
ravish@ravishPC:../sem3/is ⎇ (main)> ls -l ./prog/
total 44
-rw-rw-r-- 1 ravish ravish   353 Aug 18 12:18 a2.c
-rwxrwxr-x 1 ravish ravish 17480 Aug 18 12:24 a2.out
-rwxrwxr-x 1 ravish ravish 17568 Aug 18 12:42 a2_pro.out
ravish@ravishPC:../sem3/is ⎇ (main)>
```

running with safe size input

```
ravish@ravishPC:../sem3/is ⎇ (main)> ./prog/a2_pro.out "Testing"
Buffered content : Testing
ravish@ravishPC:../sem3/is ⎇ (main)>
```

running with very large input

```
ravish@ravishPC:../sem3/is ⎇ (main)> ./prog/a2_pro.out "Testing program
with a
 overflowing input"
Buffered content : Testing program with a overflowing input
*** stack smashing detected ***: terminated
Aborted (core dumped)
ravish@ravishPC:../sem3/is ⎇ (main)>
```

opening program binary in gdb

```
ravish@ravishPC:../sem3/is ⎇ (main)> gdb ./prog/a2_pro.out
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later
<http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./prog/a2_pro.out...
```

running program with long input

```
(gdb) run "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
Starting program: /home/ravish/work/sem3/is/prog/a2_pro.out
"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Buffered content : AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
*** stack smashing detected ***: terminated

Program received signal SIGABRT, Aborted.
```

```
__pthread_kill_implementation (no_tid=0, signo=6, threadid=<optimized out>)
    at ./nptl/pthread_kill.c:44
warning: 44     ./nptl/pthread_kill.c: No such file or directory
```

checking backtrack

```
(gdb) bt
#0  __pthread_kill_implementation (no_tid=0, signo=6,
    threadid=<optimized out>) at ./nptl/pthread_kill.c:44
#1  __pthread_kill_internal (signo=6, threadid=<optimized out>)
    at ./nptl/pthread_kill.c:78
#2  __GI___pthread_kill (threadid=<optimized out>, signo=signo@entry=6)
    at ./nptl/pthread_kill.c:89
#3  0x00007ffff7c4527e in __GI_raise (sig=sig@entry=6)
    at ../sysdeps/posix/raise.c:26
#4  0x00007ffff7c288ff in __GI_abort () at ./stdlib/abort.c:79
#5  0x00007ffff7c297b6 in __libc_message_impl (
    fmt=fmt@entry=0x7ffff7dce765 "*** %s ***: terminated\n")
    at ../sysdeps/posix/libc_fatal.c:134
#6  0x00007ffff7d36d49 in __GI___fortify_fail (
    msg=msg@entry=0x7ffff7dce77d "stack smashing detected")
    at ./debug/fortify_fail.c:24
#7  0x00007ffff7d37fd4 in __stack_chk_fail () at
./debug/stack_chk_fail.c:24
#8  0x00000000004011d8 in testing_function (
    input=0x7fffffffdb37 'A' <repeats 40 times>) at ./prog/a2.c:9
#9  0x0000000000401200 in main (
    argc=<error reading variable: Cannot access memory at address
0x4141414141413d>,
    args=<error reading variable: Cannot access memory at address
0x4141414141414131>) at ./prog/a2.c:13
Backtrace stopped: Cannot access memory at address 0x4141414141414149
```

checking debug frame

```
(gdb) info frame
Stack level 0, frame at 0x7fffffffd340:
 rip = 0x7ffff7c9ec0c in __pthread_kill_implementation
    (./nptl/pthread_kill.c:44); saved rip = 0x7ffff7c4527e
 inlined into frame 1
 source language c.
 Arglist at unknown address.
 Locals at unknown address, Previous frame's sp in rsp
```

checking memory space

```
(gdb) x/40gx $rsp
0x7fffffffd2f0: 0x0000000000000000      0x49fb21b93344d100
0x7fffffffd300: 0x0000000000000000      0x0000000000000006
0x7fffffffd310: 0x00007ffff7fa3740      0x00007fffffffd480
0x7fffffffd320: 0x00007fffffffd480      0x00007fffffffd480
0x7fffffffd330: 0x00007fffffffd350      0x00007ffff7c4527e
0x7fffffffd340: 0x0000000000000000      0x00007ffff7e04b50
0x7fffffffd350: 0x00007fffffffd410      0x00007ffff7c288ff
0x7fffffffd360: 0x0000000000000020      0x0000000000000003
0x7fffffffd370: 0x0000000000000000      0x0000000000000002
0x7fffffffd380: 0x00007fffffffd3b0      0x00007fffffffd3b0
0x7fffffffd390: 0x00007fffffffd430      0x000000000000003c
0x7fffffffd3a0: 0x0000000000000004      0x00007ffff7e045c0
0x7fffffffd3b0: 0x6465726566667542      0x746e65746e6f6320
0x7fffffffd3c0: 0x0000000000000002      0x800000000000000e
0x7fffffffd3d0: 0x0000000000000000      0x00007ffff7d2a8d1
0x7fffffffd3e0: 0x0000000000000000      0x49fb21b93344d100
0x7fffffffd3f0: 0x0000000000000000      0x49fb21b93344d100
0x7fffffffd400: 0x0000000000000000      0x0000000000001000
0x7fffffffd410: 0x00007fffffffd530      0x00007ffff7c297b6
0x7fffffffd420: 0x00007fffffffd450      0x000000000000002c
```

checking register values (it can be seen that "AAAAAAAA..." is not in the rbp register)

```
(gdb) info registers rip rbp rsp
rip            0x7ffff7c9ec0c      0x7ffff7c9ec0c <__GI___pthread_kill+284>
rbp            0x7fffffffd330      0x7fffffffd330
rsp            0x7fffffffd2f0      0x7fffffffd2f0
(gdb)
```